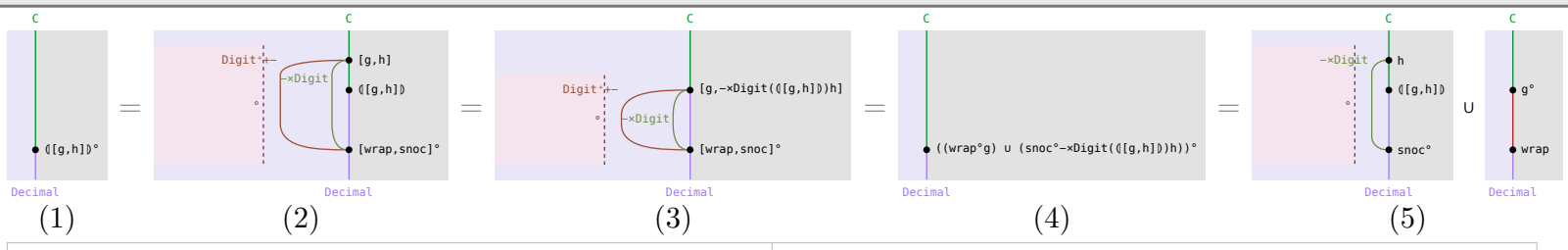
# Companion to Bird & de Moor, Algebra of Programming

# §6. Recursive Programs

## §6.1. Digits of a number

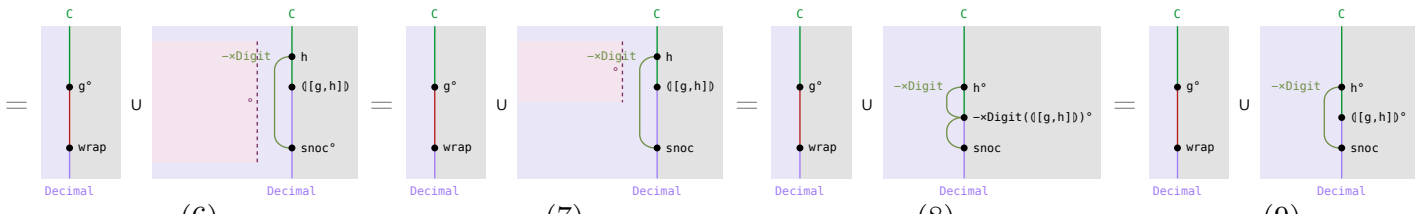
**`⦇[g,h]⦈° = (g° wrap) ∪ (h° −×Digit(⦇[g,h]⦈°) snoc)`** (6.1a)

the converse of a fold over decimals undoes the algebra `[g,h]` and then either `wrap`, or the fold's converse on the front and `snoc`




(1) (2) (3) (4) (5)

| | | |
|---|---|---|
| 1 | `⦇[g,h]⦈°` | `⦇φ⦈ = [wrap,snoc]° Digit⁺+−(−×Digit(⦇φ⦈)) φ` |
| 2 | `= ([wrap,snoc]° Digit⁺+−(−×Digit(⦇[g,h]⦈))) [g,h])°` | `(R + S) [P,Q] = [R P,S Q]` |
| 3 | `= ([wrap,snoc]° [g,−×Digit(⦇[g,h]⦈) h])°` | `[U,V]° [R,S] = (U° R) ∪ (V° S)` |
| 4 | `= ((wrap° g) ∪ (snoc° −×Digit(⦇[g,h]⦈) h))°` | `(R ∪ S)° = S° ∪ R°` |
| 5 | `= (snoc° −×Digit(⦇[g,h]⦈) h)° ∪ (wrap° g)°` | `R ∪ S = S ∪ R` |




(6) (7) (8) (9)

| | | |
|---|---|---|
| 6 | `= (wrap° g)° ∪ (snoc° −×Digit(⦇[g,h]⦈) h)°` | `(R S)° = S° R°` |
| 7 | `= (g° wrap) ∪ ((−×Digit(⦇[g,h]⦈) h)° snoc)` | `(R h)° = h° R°` |
| 8 | `= (g° wrap) ∪ (h° −×Digit(⦇[g,h]⦈)° snoc)` | `−×Digit(R)° = −×Digit(R°)` |
| 9 | `= (g° wrap) ∪ (h° −×Digit(⦇[g,h]⦈°) snoc)` | |

`val° = (embed° wrap) ∪ (op° −×Digit(val°) snoc)`

a number is read back into digits either as one nonzero digit, or by splitting off its last digit and reading back the rest

(6.1b)

| | |
|---|---|
| 1 | `op°(m,p) ⇔ π₁(p) = m/10 ∧ π₂(p) = m % 10`<br>`op°` splits a number into its quotient and remainder by 10 |
| 2 | `(∃p. 0 < π₁(p) ∧ op°(m,p)) ⇔ 10 ≤ m`<br>`op°` gives a pair with a nonzero first component exactly at the numbers with two or more digits |
| 3 | `(∃d. embed°(m,d)) ⇔ 0 < m ∧ m < 10`<br>`embed°` gives a digit exactly at the one-digit numbers |

## §6.2. Least fixed points

(6.2a)

`Monotonic(φ) ≜ ∀X. ∀Y. X ⊑ Y ⇒ φ(X) ⊑ φ(Y)` — φ maps the hom-set `A↛B` into itself with `X⊑Y⇒φ(X)⊑φ(Y)`.

`(μX : φ(X))` is the least `X : A↛B` with `φ(X)⊑X`.

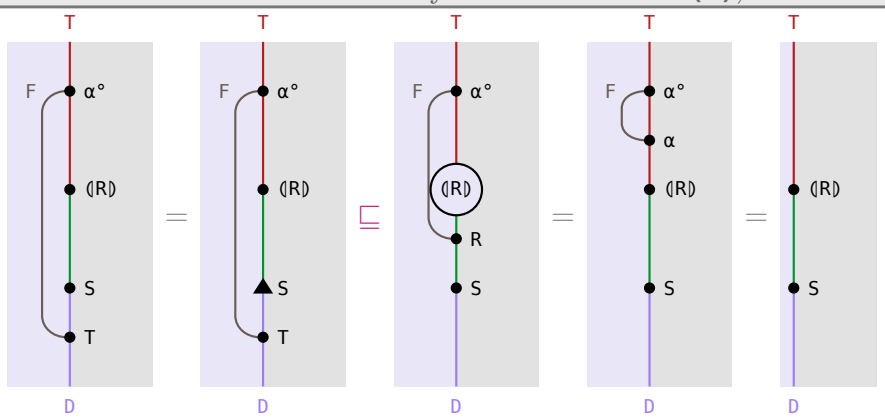
(6.2b)

| | the law | what it says |
|---|---|---|
| 1 | `φ(Y) ⊑ Y ⇒ (μX : φ(X)) ⊑ Y` | to bound `(μX : φ(X))` above, exhibit one `Y` the body does not grow past — the half §Section 6.3. and every chapter after it uses |
| 2 | `Monotonic(φ) ⇒ φ(μX : φ(X)) = (μX : φ(X))` | **Knaster–Tarski**: the least solution of `φ(X)⊑X` already solves `φ(X)=X`, so the least prefix point and the least fixed point are one relation |
| 3 | `α° F(X) R ⊑ X ⇒ (R) ⊑ X` (6.2) | a relation `X` that contains one unfolding of the fold's recursion at `X` contains the fold |
| 4 | `X ⊑ α° F(X) R ⇒ X ⊑ (R)` (6.3) | a relation `X` contained in one unfolding of the fold's recursion at `X` is contained in the fold |

(6.2c)

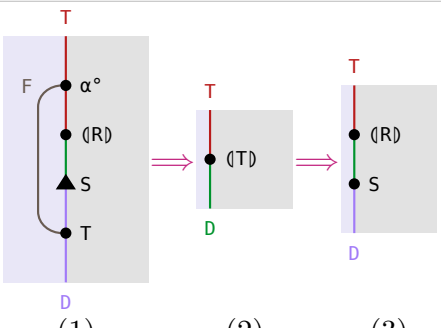
**`F(S) T ⊑ R S`**
**`⇒ (T) ⊑ (R) S`**

if **S** followed by **R** absorbs **T** after **F(S)**, the fold of **T** is below the fold of **R** followed by **S**




(1) (2) (3) (4) (5)

| | | |
|---|---|---|
| 1 | `α° F((R) S) T` | |
| 2 | `= α° F((R)) F(S) T` | `F(S) T ⊑ R S` |
| 3 | `⊑ α° F((R)) R S` | `α (R) = F((R)) R` |
| 4 | `= α° α (R) S` | `α° α = 𝟙` |
| 5 | `= (R) S` | |

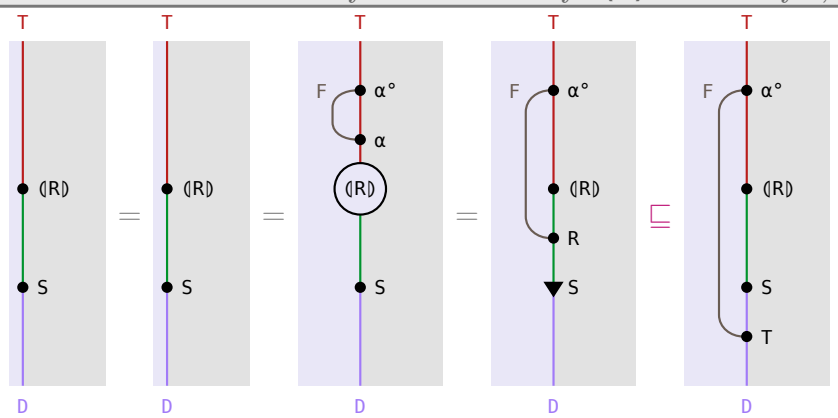



(1) (2) (3)

| | | |
|---|---|---|
| 1 | `F(S) T ⊑ R S` | `F(S) T ⊑ R S`<br>`⇒ α° F((R) S) T ⊑ (R) S` |
| 2 | `⟹ α° F((R) S) T ⊑ (R) S` | `α° F(X) T ⊑ X`<br>`⇒ (T) ⊑ X` |
| 3 | `⟹ (T) ⊑ (R) S` | |

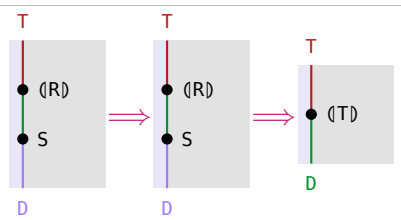
$$R\ S \sqsubseteq F(S)\ T \quad \Rightarrow (\!|R|\!)\ S \sqsubseteq (\!|T|\!) \tag{6.2d}$$

if $R$ followed by $S$ is absorbed by $F(S)$ followed by $T$, the fold of $R$ followed by $S$ is below the fold of $T$




(1) (2) (3) (4) (5)

| | | |
|---|---|---|
| 1 | $(\!|R|\!)\ S$ | |
| 2 | $= (\!|R|\!)\ S$ | $\alpha^\circ\ \alpha = \mathbb{1}$ |
| 3 | $= \alpha^\circ\ \alpha\ (\!|R|\!)\ S$ | $\alpha\ (\!|R|\!) = F(\!|R|\!)\ R$ |
| 4 | $= \alpha^\circ\ F(\!|R|\!)\ R\ S$ | $R\ S \sqsubseteq F(S)\ T$ |
| 5 | $\sqsubseteq \alpha^\circ\ F(\!|R|\!)\ S)\ T$ | |



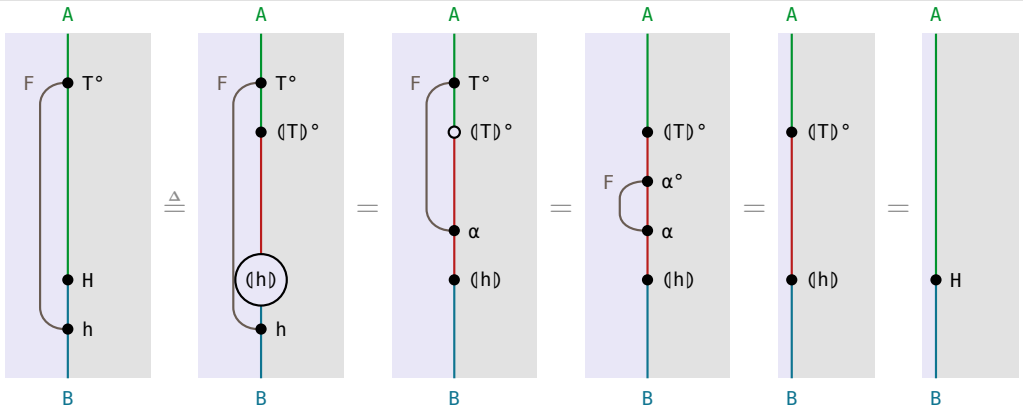

(1) (2) (3)

| | | |
|---|---|---|
| 1 | $R\ S \sqsubseteq F(S)\ T$ | $R\ S \sqsubseteq F(S)\ T$<br>$\Rightarrow (\!|R|\!)\ S \sqsubseteq \alpha^\circ\ F(\!|R|\!)\ S)\ T$ |
| 2 | $\Longrightarrow (\!|R|\!)\ S \sqsubseteq \alpha^\circ\ F(\!|R|\!)\ S)\ T$ | $X \sqsubseteq \alpha^\circ\ F(X)\ T$<br>$\Rightarrow X \sqsubseteq (\!|T|\!)$ |
| 3 | $\Longrightarrow (\!|R|\!)\ S \sqsubseteq (\!|T|\!)$ | |

## §6.3. Hylomorphisms

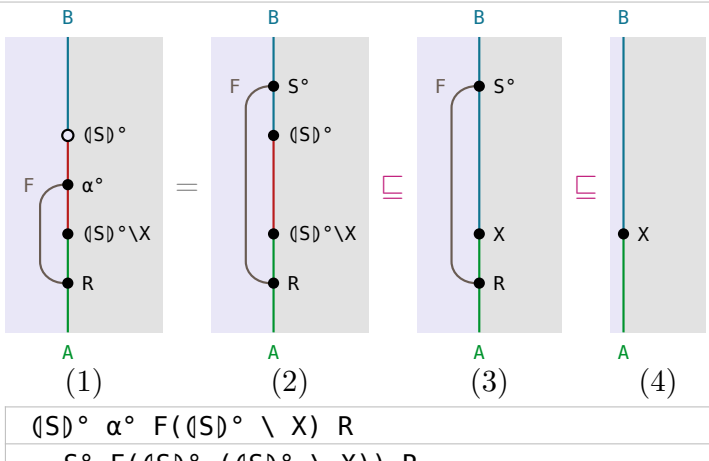
**⦇S⦈° ⦇R⦈ = (μX : S° F(X) R)** (6.3a)

hylomorphism theorem: a hylomorphism is the least fixed point of a certain recursion equation



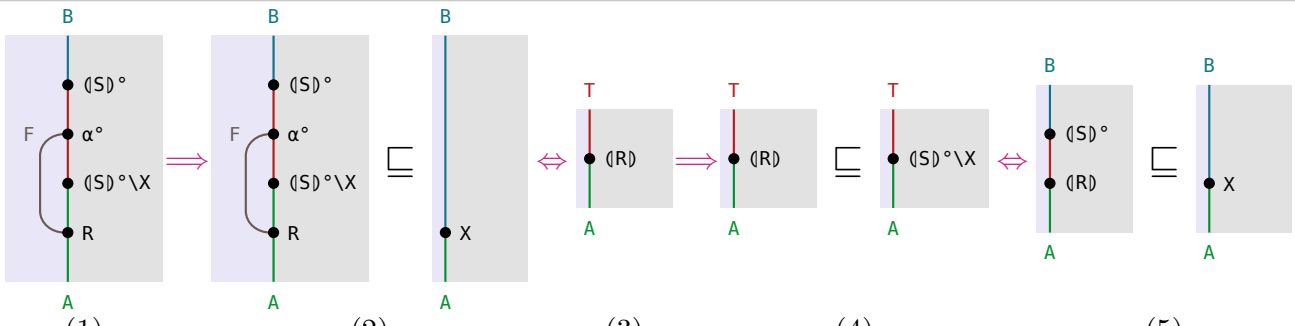

(1) (2) (3) (4) (5) (6)

| | | |
|---|---|---|
| 1 | T° F(H) h | H ≜ ⦇T⦈° ⦇h⦈ |
| 2 | ≜ T° F(⦇T⦈° ⦇h⦈) h | α ⦇h⦈ = F(⦇h⦈) h |
| 3 | = T° F(⦇T⦈°) α ⦇h⦈ | ⦇T⦈° α° = T° F(⦇T⦈°) |
| 4 | = ⦇T⦈° α° α ⦇h⦈ | α° α = 𝟙 |
| 5 | = ⦇T⦈° ⦇h⦈ | H = ⦇T⦈° ⦇h⦈ |
| 6 | = H | |




(1) (2) (3) (4)

| | | |
|---|---|---|
| 1 | ⦇S⦈° α° F(⦇S⦈° \ X) R | ⦇S⦈° α° = S° F(⦇S⦈°) |
| 2 | = S° F(⦇S⦈° (⦇S⦈° \ X)) R | S (S \ X) ⊑ X |
| 3 | ⊑ S° F(X) R | S° F(X) R ⊑ X |
| 4 | ⊑ X | |

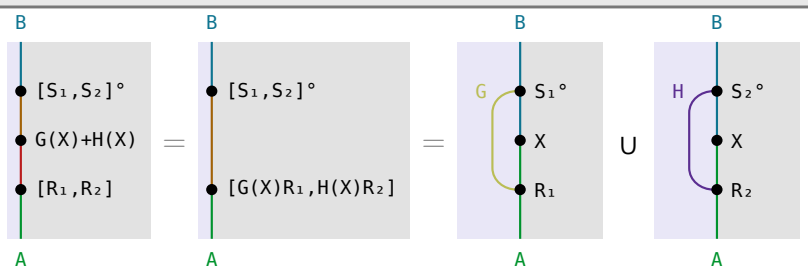



(1) (2) (3) (4) (5)

| | | |
|---|---|---|
| 1 | S° F(X) R ⊑ X | S° F(X) R ⊑ X<br>⇒ ⦇S⦈° α° F(⦇S⦈° \ X) R ⊑ X |
| 2 | ⟹ ⦇S⦈° α° F(⦇S⦈° \ X) R ⊑ X | X ⊑ S \ R ⇔ S X ⊑ R |
| 3 | ⟺ α° F(⦇S⦈° \ X) R ⊑ ⦇S⦈° \ X | α° F(X) R ⊑ X<br>⇒ ⦇R⦈ ⊑ X |
| 4 | ⟹ ⦇R⦈ ⊑ ⦇S⦈° \ X | X ⊑ S \ R ⇔ S X ⊑ R |
| 5 | ⟺ ⦇S⦈° ⦇R⦈ ⊑ X | |

**⦇[S₁,S₂]⦈° ⦇[R₁,R₂]⦈ = (μX : (S₁° G(X) R₁) ∪ (S₂° H(X) R₂))** (6.3b)

when both algebras are case splits over the same sum, the recursion runs each case on its own and unites the results




(1) (2) (3)

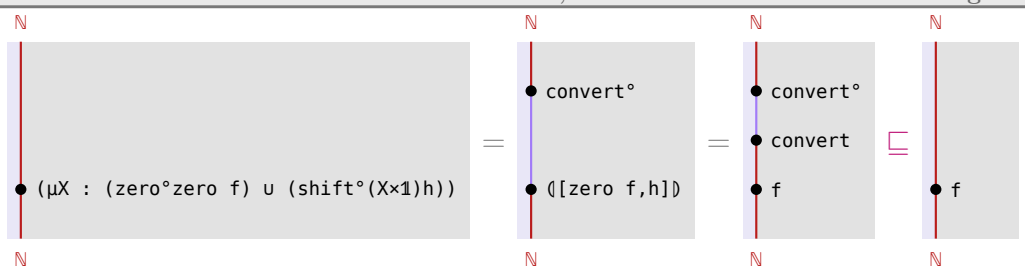
| | | |
|---|---|---|
| 1 | `[S₁,S₂]° (G(X) + H(X)) [R₁,R₂]` | `(R + S) [R₁,R₂] = [R R₁,S R₂]` |
| 2 | `= [S₁,S₂]° [G(X) R₁,H(X) R₂]` | `[S₁,S₂]° [R,S] = (S₁° R) ∪ (S₂° S)` |
| 3 | `= (S₁° G(X) R₁) ∪ (S₂° H(X) R₂)` | |

## §6.4. Fast exponentiation and modulus computation

**shift f = (f × 𝟙) h** (6.4a)

**⇒ (μX : (zero° zero f) ∪ (shift° (X × 𝟙) h)) ⊑ f**

once **f** turns **shift** into **h**, the recursion that halves the argument at each step computes **f**




| | | |
|---|---|---|
| 1 | (μX : (zero° zero f) ∪ (shift° (X × 𝟙) h)) | convert° ⦇[g,h]⦈ = (μX : (zero° g) ∪ (shift° (X × 𝟙) h)) |
| 2 | = convert° ⦇[zero f,h]⦈ | zero f = g; shift f = (f × 𝟙) h ⇒ convert f = ⦇[g,h]⦈ |
| 3 | = convert° convert f | Simple(convert) |
| 4 | ⊑ f | |

**(μX : (zero° one) ∪ (shift° (X × 𝟙) op(a))) ⊑ exp(a)** (6.4b)

the recursion that halves the exponent at each step computes **a** to the power **b**

`zero exp(a) = one`

`a` to the power `0` is `1`

`shift exp(a) = (exp(a) × 𝟙) op(a)`

`a` to the power `2n+d` is `op(a)` of `a` to the power `n` and `d`

**(μX : (zero° zero) ∪ (shift° (X × 𝟙) op(b))) ⊑ mod(b)** (6.4c)

the recursion that halves **a** at each step computes the remainder of **a** by **b**

`zero mod(b) = zero`

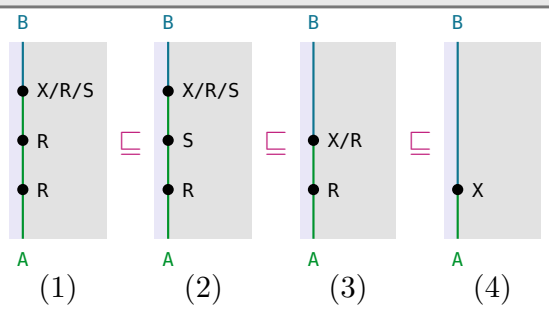
`0 mod b` is `0`

`shift mod(b) = (mod(b) × 𝟙) op(b)`

`(2a+d) mod b` is `op(b)` of `a mod b` and `d`

## §6.5. Unique fixed points

$$R^2 \sqsubseteq S\ R \quad (6.5a)$$
$$\Rightarrow (X\ /\ R\ /\ S)\ R^2 \sqsubseteq X$$
((X/R)/S) followed by RR stays below X when RR⊑SR
**inductive(S); $R^2 \sqsubseteq S\ R$**
**⇒ inductive(R)**
R is inductive when RR⊑SR for an inductive S




| | | |
|---|---|---|
| 1 | (X / R / S) $R^2$ | $R^2 \sqsubseteq S\ R$ |
| 2 | $\sqsubseteq$ (X / R / S) S R | (R / S) S ⊑ R |
| 3 | $\sqsubseteq$ (X / R) R | (X / R) R ⊑ X |
| 4 | $\sqsubseteq$ X | |

### §6.5.1. Membership

**member = ∅** (6.5.1a)
a constant relator records no elements, so its membership is empty
**member = [member,member]**
a member of F+G is a member of whichever summand the value is in
**member = ($\pi_1$ member) ∪ ($\pi_2$ member)**
a member of F×G is a member of either component

### §6.5.2. Consequences

**inductive(S member); X = S F(X) R; Y = S F(Y) R** (6.5.2a)
**⇒ X = Y**
when S member is inductive, two solutions X, Y of X=SF(X)R are equal

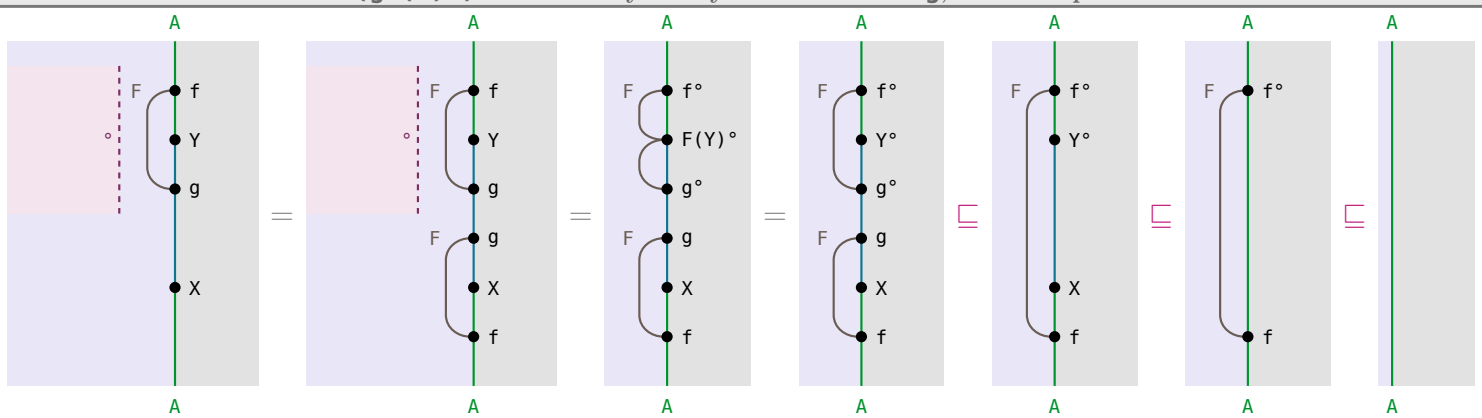
**inductive(S member); Entire(S); Entire(R); S F(X) R ⊑ X** (6.5.2b)
**⇒ Entire(X)**
when S member is inductive and S, R are entire, a pre-fixed point of X↦SF(X)R is entire

**inductive(g member); Map(g); Map(f); X = g F(X) f**
**⇒ Map(X)** (6.5.2c)

when **g member** is inductive and **f**, **g** are maps, the solution of **X=gF(X)f** is a map

**X = g F(X) f; Simple(g); Simple(f); Y° X ⊑ 𝟙**
**⇒ (g F(Y) f)° X ⊑ 𝟙**

**(gF(Y)f)°** followed by **X** stays below **𝟙** when **g**, **f** are simple and **Y°X⊑𝟙**

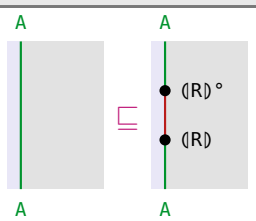



(1) (2) (3) (4) (5) (6) (7)

| | | |
|---|---|---|
| 1 | (g F(Y) f)° X | X = g F(X) f |
| 2 | = (g F(Y) f)° g F(X) f | (R S)° = S° R° |
| 3 | = f° F(Y)° g° g F(X) f | PreservesRecip(F) |
| 4 | = f° F(Y°) g° g F(X) f | Simple(g) |
| 5 | ⊑ f° F(Y° X) f | Y° X ⊑ 𝟙 |
| 6 | ⊑ f° f | Simple(f) |
| 7 | ⊑ 𝟙 | |

**inductive(R° member); 𝟙 ⊑ R° R**
**⇒ 𝟙 ⊑ (R)° (R)** (6.5.2d)

when **R°member** is inductive and **R** is surjective, the fold **(R)** is surjective

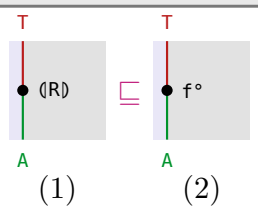



(1) (2)

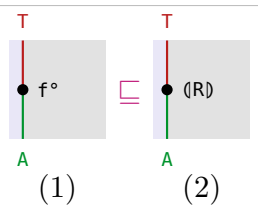
| | | |
|---|---|---|
| 1 | 𝟙 | inductive(S member); Entire(S); Entire(R); S F(X) R ⊑ X ⇒ Entire(X) at S=R° |
| 2 | ⊑ (R)° (R) | |

**Map(f); R f ⊑ F(f) α; 𝟙 ⊑ R° R**
**⇒ f° = ⦇R⦈** (6.5.2e)

a map f with Rf⊑F(f)α, for a surjective R, has the fold ⦇R⦈ as its converse



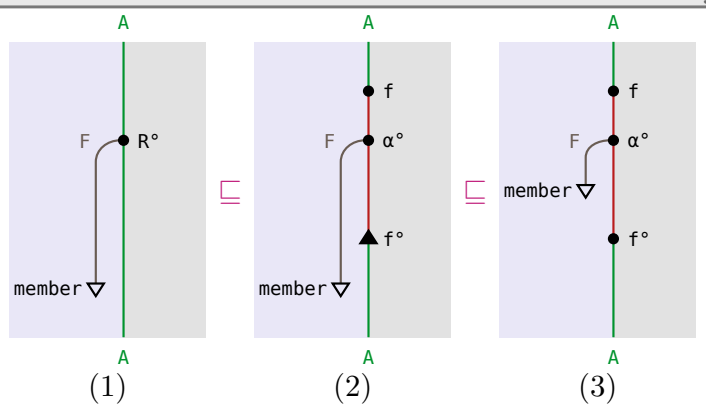

| | | |
|---|---|---|
| 1 | ⦇R⦈ | shunting f, fusion, Rf⊑F(f)α |
| 2 | ⊑ f° | |




| | | |
|---|---|---|
| 1 | f° | inductive(R° member); 𝟙 ⊑ R° R<br>⇒ 𝟙 ⊑ ⦇R⦈° ⦇R⦈ and ⦇R⦈f⊑𝟙 |
| 2 | ⊑ ⦇R⦈ | |

**Map(f); R f ⊑ F(f) α**
**⇒ R° member ⊑ f α° member f°** (6.5.2f)

R°member is below α°member conjugated by f, so it is inductive when α°member is




| | | |
|---|---|---|
| 1 | R° member | Map(f); R f ⊑ F(f) α<br>⇒ R° ⊑ f α° F(f°) |
| 2 | ⊑ f α° F(f°) member | F(R) member ⊑ member R |
| 3 | ⊑ f α° member f° | |

# §6.6. Sorting by selection

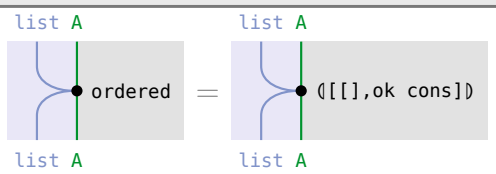
## §6.6.1. The function sort

(6.6.1a)

| | definition | type | meaning |
|---|---|---|---|
| 1 | `ordered[] = true`<br>`ordered([a]#xs) = (∀x. x ∈ xs ⇒ R(a,x)) ∧ ordered(xs)` | `[A]→ [A]` | every element is R-below each element after it |
| 2 | `(a,xs) ok q ⇔ (a,xs) = q ∧ (∀x. x ∈ xs ⇒ R(a,x))` | `A×[A]→ A×[A]` | the coreflexive passing `(a, x)` when `a` is R-below every element of `x` |

**`ordered = ⦇[[],ok cons]⦈`** (6.6.1b)

a list is sorted exactly when the fold that rebuilds it passes **ok** at every **cons**, i.e. each head is **R**-below every element after it

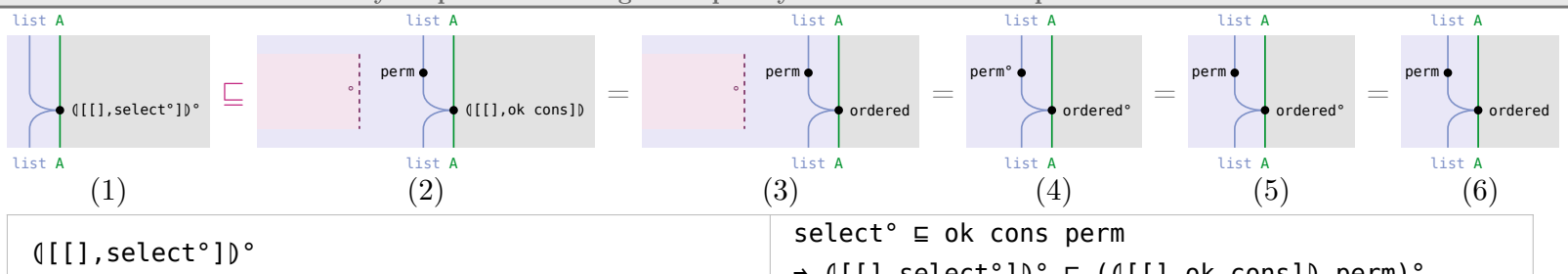



| | | |
|---|---|---|
| 1 | `ordered` | fold uniqueness |
| 2 | `= ⦇[[],ok cons]⦈` | |

## §6.6.2. Selection sort

**`select° ⊑ ok cons perm`**
**`⇒ ⦇[[],select°]⦈° ⊑ perm ordered`** (6.6.2a)

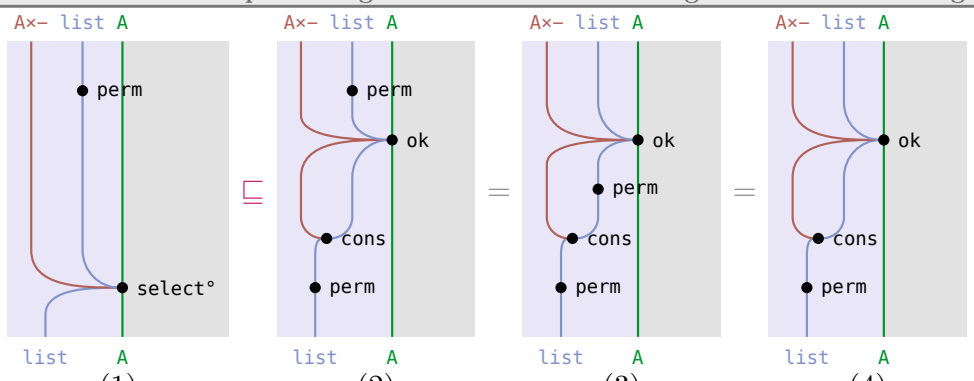
every output of unfolding the input by **select** is a sorted permutation of it




| | | |
|---|---|---|
| 1 | `⦇[[],select°]⦈°` | `select° ⊑ ok cons perm`<br>`⇒ ⦇[[],select°]⦈° ⊑ (⦇[[],ok cons]⦈ perm)°` |
| 2 | `⊑ (⦇[[],ok cons]⦈ perm)°` | (6.6.1b) `ordered = ⦇[[],ok cons]⦈` |
| 3 | `= (ordered perm)°` | `(ordered perm)° = perm° ordered°` |
| 4 | `= perm° ordered°` | `perm° = perm` |
| 5 | `= perm ordered°` | `ordered° = ordered` |
| 6 | `= perm ordered` | |

**`select° ⊑ ok cons perm`**
**`⇒ (𝟙 × perm) select° ⊑ ok cons perm`** (6.6.2b)

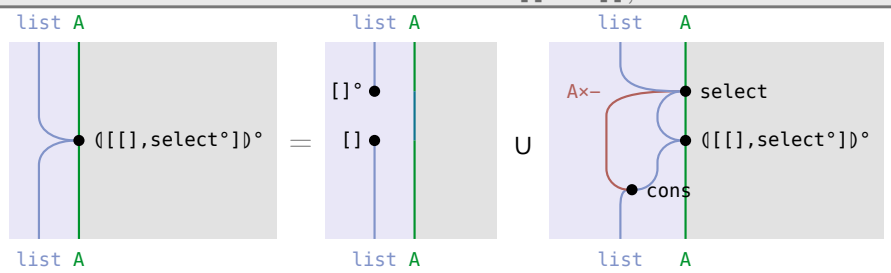
permuting the tail and then undoing **select** lands among the **ok** conses of a permutation




| | | |
|---|---|---|
| 1 | `(𝟙 × perm) select°` | `select° ⊑ ok cons perm` |
| 2 | `⊑ (𝟙 × perm) ok cons perm` | `ok (𝟙 × perm) = (𝟙 × perm) ok` |
| 3 | `= ok (𝟙 × perm) cons perm` | `cons perm = (𝟙 × perm) cons perm` |
| 4 | `= ok cons perm` | |

**(𝕝[[],select°]𝕝)° = ([]° []) ∪ (select (𝟙 × 𝕝[[],select°]𝕝°) cons)** (6.6.2c)

selection sort returns `[]` on `[]`, and otherwise selects `(a,y)`, sorts `y` and conses `a` back on



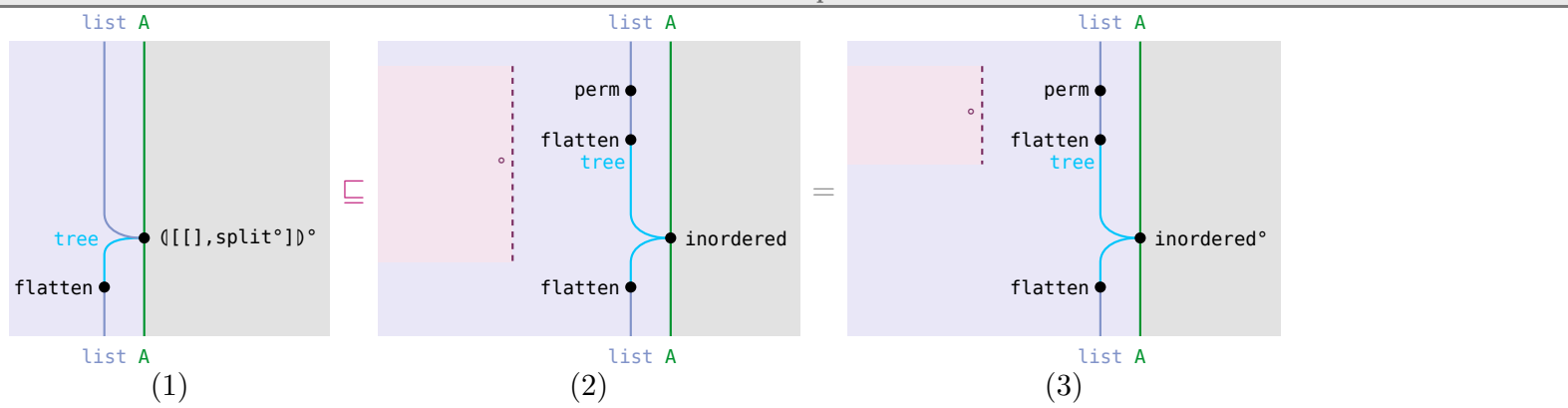

(1) (2)

| | | |
|---|---|---|
| 1 | `𝕝[[],select°]𝕝°` | unfolding the converse of a fold |
| 2 | `= ([]° []) ∪ (select (𝟙 × 𝕝[[],select°]𝕝°) cons)` | |

## §6.6.3. Quicksort

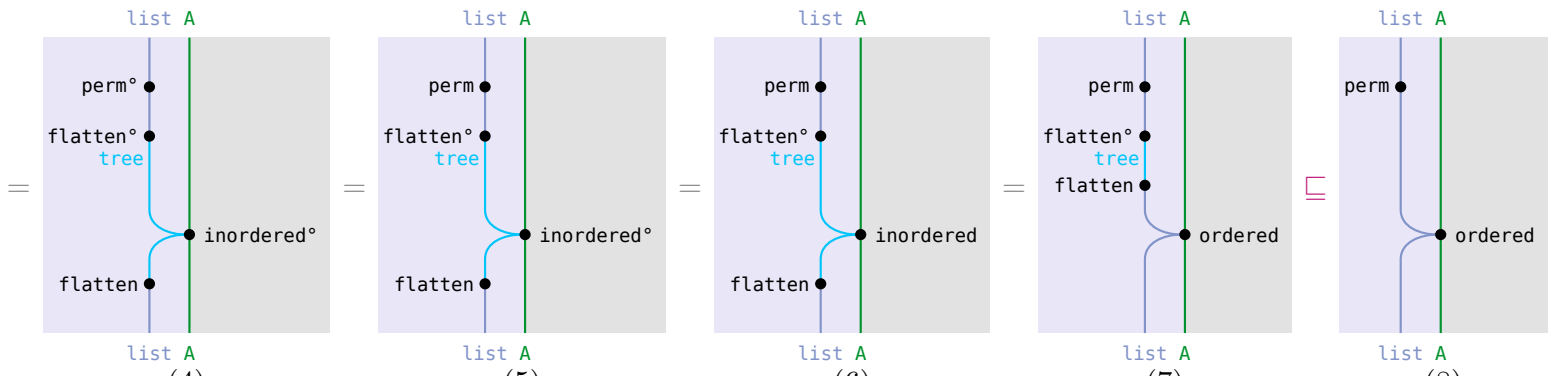
**(∀a. ∀b. ∀c. R(a,b) ⇒ R(b,c) ⇒ R(a,c)); split° ⊑ check' join perm ⇒ 𝕝[[],split°]𝕝° flatten ⊑ perm ordered** (6.6.3a)

every output of unfolding the input by `split` into a tree and flattening that tree is a sorted permutation of the input




(1) (2) (3)

| | | |
|---|---|---|
| 1 | `𝕝[[],split°]𝕝° flatten` | `split° ⊑ check' join perm ⇒ 𝕝[[],split°]𝕝° flatten ⊑ (inordered flatten perm)° flatten` |
| 2 | `⊑ (inordered flatten perm)° flatten` | `(inordered S)° = S° inordered°` |
| 3 | `= (flatten perm)° inordered° flatten` | `(flatten perm)° = perm° flatten°` |

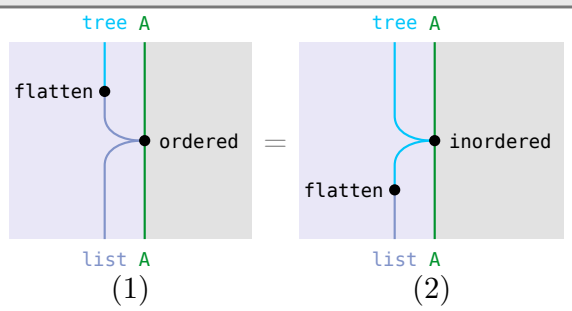



(4) (5) (6) (7) (8)

| | | |
|---|---|---|
| 4 | `= perm° flatten° inordered° flatten` | `perm° = perm` |
| 5 | `= perm flatten° inordered° flatten` | `inordered° = inordered` |
| 6 | `= perm flatten° inordered flatten` | (6.6.3b) `(∀a. ∀b. ∀c. R(a,b) ⇒ R(b,c) ⇒ R(a,c)) ⇒ flatten ordered = inordered flatten` |
| 7 | `= perm flatten° flatten ordered` | `flatten° flatten ⊑ 𝟙` |
| 8 | `⊑ perm ordered` | |

**(∀a. ∀b. ∀c. R(a,b) ⇒ R(b,c) ⇒ R(a,c))**
**⇒ flatten ordered = inordered flatten** (6.6.3b)

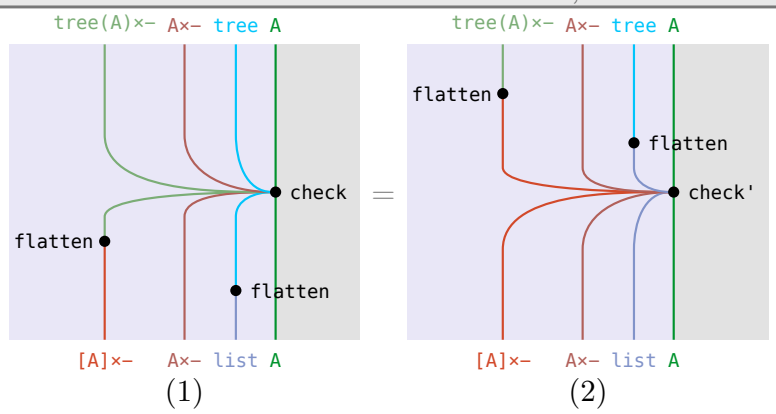
flattening a tree and then testing the list for order is the same as testing the tree with **inordered** and then flattening




| | | |
|---|---|---|
| 1 | `flatten ordered` | R transitive: a flattened tree is ordered iff every node passes `check` |
| 2 | `= inordered flatten` | |

**check (flatten × (𝟙 × flatten)) = (flatten × (𝟙 × flatten)) check'** (6.6.3c)

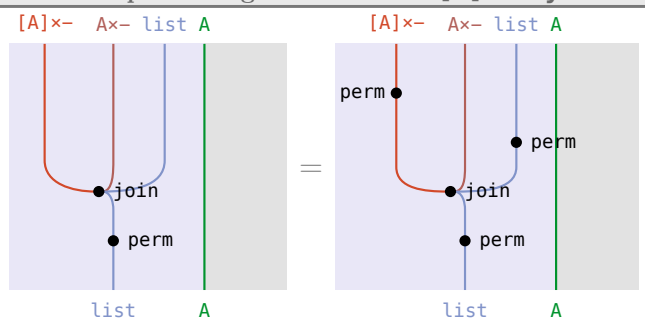
testing a fork with **check** and then flattening both subtrees equals flattening both subtrees and then testing with **check'**, since **intree** of a tree is **inlist** of its flattening




| | | |
|---|---|---|
| 1 | `check (flatten × (𝟙 × flatten))` | `b intree x ⇔ b inlist flatten(x)` |
| 2 | `= (flatten × (𝟙 × flatten)) check'` | |

**join perm = (perm × (𝟙 × perm)) join perm** (6.6.3d)

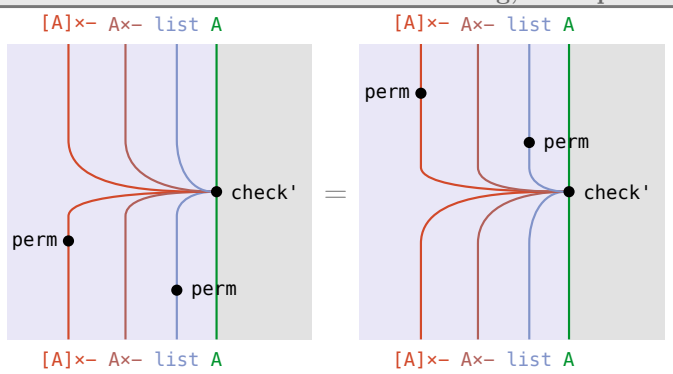
permuting the list **x ++ [a] ++ y** is the same as first permuting **x** and **y** and then permuting the joined list




| | | |
|---|---|---|
| 1 | `join perm` | `perm` is transitive and closed under `++` |
| 2 | `= (perm × (𝟙 × perm)) join perm` | |

**check' (perm × (𝟙 × perm)) = (perm × (𝟙 × perm)) check'** (6.6.3e)

testing a fork of lists with **check'** and then permuting both parts equals permuting both parts and then testing, since permuting a list leaves its members unchanged

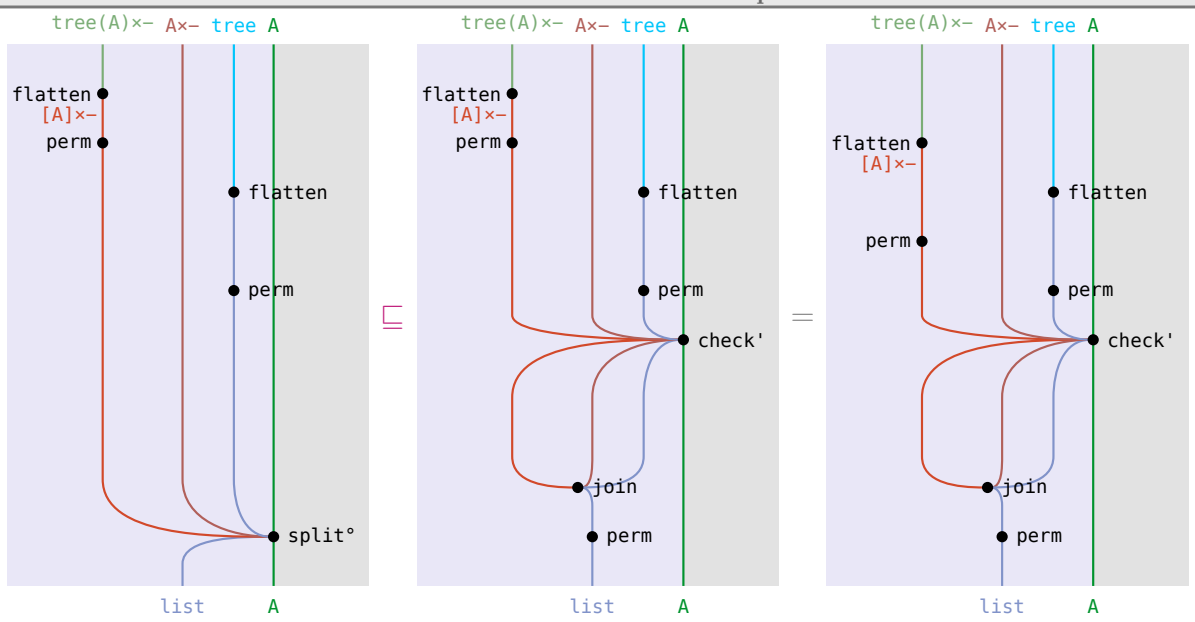



```
1  check' (perm × (𝟙 × perm))          b inlist x ⇔ b inlist perm(x)
2  = (perm × (𝟙 × perm)) check'
```

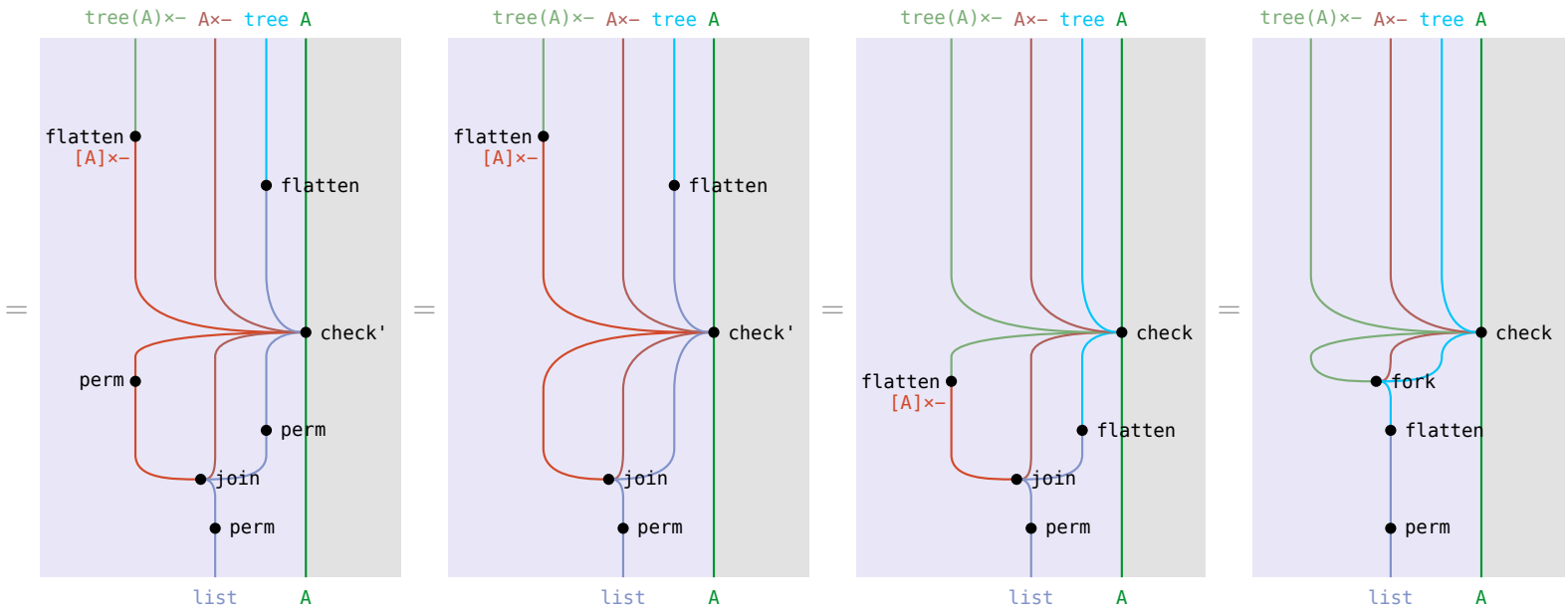
**split° ⊑ check' join perm** **(6.6.3f)**

**⇒ ((flatten perm) × (𝟙 × (flatten perm))) split° ⊑ check fork flatten perm**

undoing **split** and then flattening and permuting both parts lands among the **check**ed forks whose flattening is permuted




| | | |
|---|---|---|
| 1 | ((flatten perm) × (𝟙 × (flatten perm))) split° | split° ⊑ check' join perm |
| 2 | ⊑ ((flatten perm) × (𝟙 × (flatten perm))) check' join perm | |
| 3 | = (flatten × (𝟙 × flatten)) (perm × (𝟙 × perm)) check' join perm | (6.6.3e) check' (perm × (𝟙 × perm)) = (perm × (𝟙 × perm)) check' |

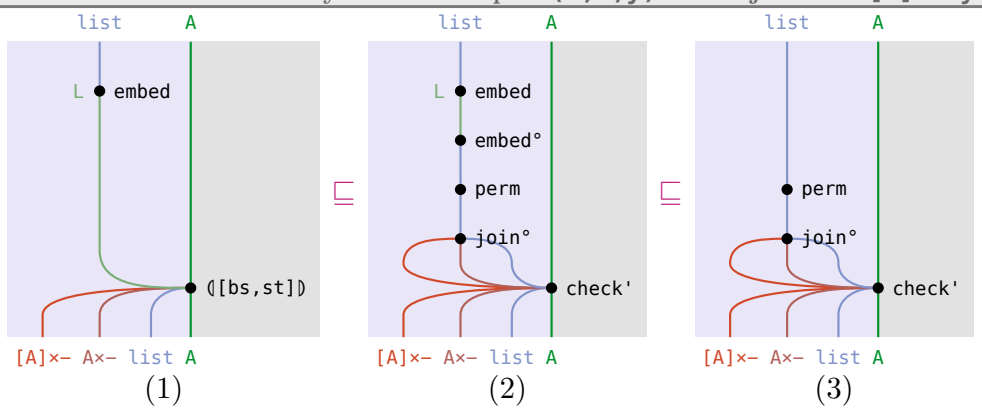



| | | |
|---|---|---|
| 4 | = (flatten × (𝟙 × flatten)) check' (perm × (𝟙 × perm)) join perm | (6.6.3d) join perm = (perm × (𝟙 × perm)) join perm |
| 5 | = (flatten × (𝟙 × flatten)) check' join perm | (6.6.3c) check (flatten × (𝟙 × flatten)) = (flatten × (𝟙 × flatten)) check' |
| 6 | = check (flatten × (𝟙 × flatten)) join perm | fork flatten = (flatten × (𝟙 × flatten)) join |
| 7 | = check fork flatten perm | |

**`bs ⊑ τ perm join° check'; (𝟙 × (perm join° check')) st ⊑ cons perm join° check'`** (6.6.3g)

**`⇒ embed (|[bs,st]|) ⊑ perm join° check'`**

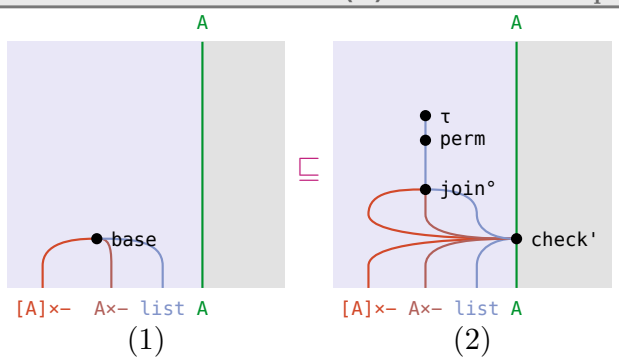
turning a list into a non-empty list and folding it with any `[bs, st]` meeting the two conditions below gives only **check**ed triples `(x,a,y)` whose join `x ⧺ [a] ⧺ y` is a permutation of the list




| | | |
|---|---|---|
| 1 | `embed (\|[bs,st]\|)` | `bs ⊑ τ perm join° check'; (𝟙 × (perm join° check')) st ⊑ cons perm join° check' ⇒ embed (\|[bs,st]\|) ⊑ embed embed° perm join° check'` |
| 2 | `⊑ embed embed° perm join° check'` | `embed embed° perm join° check' ⊑ perm join° check'` |
| 3 | `⊑ perm join° check'` | |

**`base ⊑ τ perm join° check'`** (6.6.3h)

**`base(a)`** is a **check**ed triple whose join is a permutation of the one-element list `[a]`

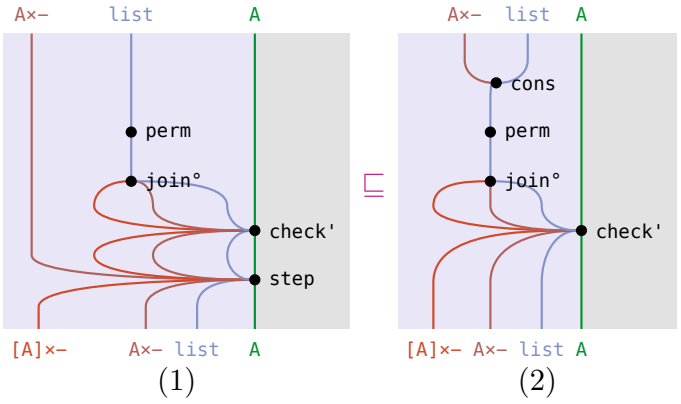



| | | |
|---|---|---|
| 1 | `base` | `[] ⧺ [a] ⧺ [] = [a]`, and `check'` holds on empty lists |
| 2 | `⊑ τ perm join° check'` | |

**`connected(R)`** (6.6.3i)

**`⇒ (𝟙 × (perm join° check')) step ⊑ cons perm join° check'`**

if **`(x,b,y)`** is a **check**ed triple for a permutation of **`l`**, then **`step(a,(x,b,y))`** is a **check**ed triple for a permutation of **`[a] ⧺ l`**

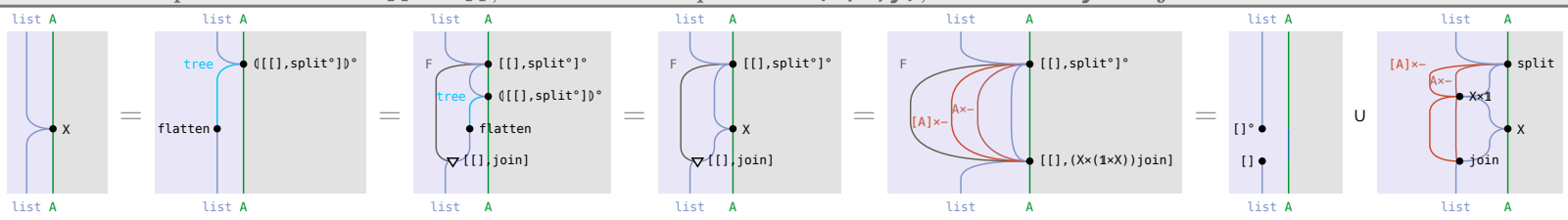



| | | |
|---|---|---|
| 1 | `(𝟙 × (perm join° check')) step` | `aRb` puts `a` in front of `x`; otherwise `bRa` and `a` goes in front of `y` |
| 2 | `⊑ cons perm join° check'` | |

**X = ◖[[[],split°]◗° flatten** (6.6.3j)

**⇒ X = ([]° []) ∪ (split (X × (𝟙 × X)) join)**

quicksort returns **[]** on **[]**, and otherwise splits into **(x,a,y)**, sorts **x** and **y** and joins them around **a**



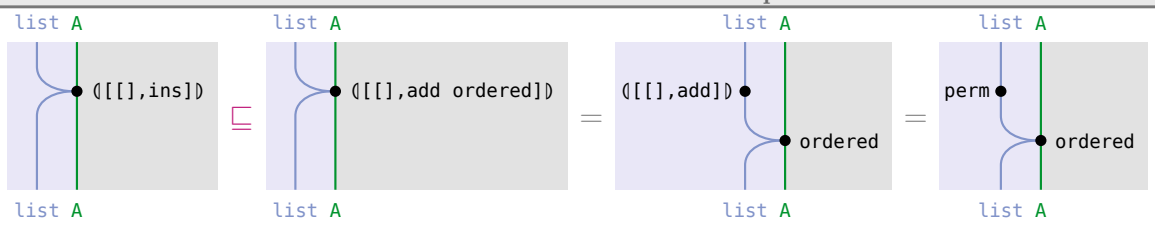

(1) (2) (3) (4) (5) (6)

| | | |
|---|---|---|
| 1 | X | X = ◖[[[],split°]◗° flatten |
| 2 | = ◖[[[],split°]◗° flatten | T° F(H) h = H |
| 3 | = [[],split°]° F(◖[[[],split°]◗° flatten) [[],join] | X = ◖[[[],split°]◗° flatten |
| 4 | = [[],split°]° F(X) [[],join] | F(X) [T,join] = [T,(X × (𝟙 × X)) join] |
| 5 | = [[],split°]° [[],(X × (𝟙 × X)) join] | [U,V]° [R,S] = (U° R) ∪ (V° S) |
| 6 | = ([]° []) ∪ (split (X × (𝟙 × X)) join) | |

**(𝟙 × ordered) ins ⊑ add ordered** (6.6.3k)

**⇒ ◖[[[],ins]◗ ⊑ perm ordered**

for any **ins** that, on an ordered list, returns only ordered results of **add**, folding with **[nil, ins]** gives only sorted permutations



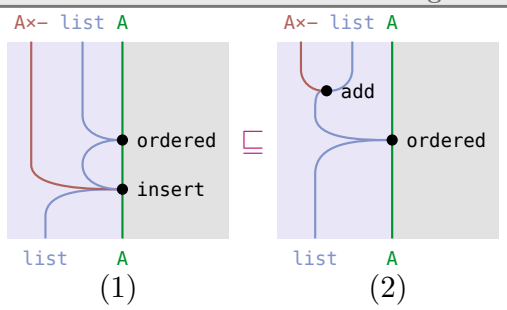

(1) (2) (3) (4)

| | | |
|---|---|---|
| 1 | ◖[[[],ins]◗ | (𝟙 × ordered) ins ⊑ add ordered<br>⇒ ◖[[[],ins]◗ ⊑ ◖[[[],add ordered]◗ |
| 2 | ⊑ ◖[[[],add ordered]◗ | ◖[[[],add]◗ ordered = ◖[[[],add ordered]◗ |
| 3 | = ◖[[[],add]◗ ordered | perm = ◖[[[],add]◗ |
| 4 | = perm ordered | |

**connected(R); (∀a. ∀b. ∀c. R(a,b) ⇒ R(b,c) ⇒ R(a,c))** (6.6.3l)

**⇒ (𝟙 × ordered) insert ⊑ add ordered**

inserting **a** into an ordered list puts **a** somewhere in it and keeps it ordered




(1) (2)

| | | |
|---|---|---|
| 1 | (𝟙 × ordered) insert | insert splices a in, and R transitive and connected |
| 2 | ⊑ add ordered | |

## §6.7. Closure

$$R^* = (\mu X : \mathbb{1} \cup (X\ R)) \tag{6.7a}$$

closing $R$ by composing it on the left and closing it by composing on the right give the same relation $R^*$

$$\mathbb{1} \sqsubseteq X;\ X^2 \sqsubseteq X \Rightarrow R \sqsubseteq X \Leftrightarrow R^* \sqsubseteq X \tag{6.7b}$$

$X$ contains $R$ exactly when it contains $R^*$

$$\mathbb{1} \sqsubseteq R^* \tag{6.7c}$$

$R^*$ is reflexive

| | | |
|---|---|---|
| 1 | $\mathbb{1}$ | $R \sqsubseteq R \cup S$ |
| 2 | $\sqsubseteq\ \mathbb{1} \cup (R\ R^*)$ | $\mathbb{1} \cup (R\ R^*) = R^*$ |
| 3 | $=\ R^*$ | |

$$R \sqsubseteq R^* \tag{6.7d}$$

$R^*$ contains $R$

| | | |
|---|---|---|
| 1 | $R$ | |
| 2 | $=\ R$ | (6.7c) $\mathbb{1} \sqsubseteq R^*$ |
| 3 | $\sqsubseteq\ R\ R^*$ | $R\ R^* \sqsubseteq R^*$ |
| 4 | $\sqsubseteq\ R^*$ | |

$$(\mathbb{1} \cup (R\ (R^* \,/\, R^*)))\ R^* \sqsubseteq R^* \tag{6.7e}$$

$R^*/R^*$ is a prefixed point of $X \mapsto \mathbb{1} \cup RX$, stated through division

$$R^{*2} \sqsubseteq R^*$$

$R^*$ is transitive, because $R^*/R^*$ is a prefixed point of $X \mapsto \mathbb{1} \cup RX$

| | | |
|---|---|---|
| 1 | $(\mathbb{1} \cup (R\ (R^* \,/\, R^*)))\ R^*$ | $(S \cup T)\ R = (S\ R) \cup (T\ R)$ |
| 2 | $=\ (R^*) \cup (R\ (R^* \,/\, R^*)\ R^*)$ | |
| 3 | $=\ R^* \cup (R\ (R^* \,/\, R^*)\ R^*)$ | $(X \,/\, X)\ X \sqsubseteq X$ |
| 4 | $\sqsubseteq\ R^* \cup (R\ R^*)$ | $R\ R^* \sqsubseteq R^*$ |
| 5 | $\sqsubseteq\ R^*$ | |

$$\mathbb{1} \sqsubseteq X;\ X^2 \sqsubseteq X;\ R \sqsubseteq X \Rightarrow \mathbb{1} \cup (R\ X) \sqsubseteq X$$

a reflexive transitive $X$ containing $R$ is a prefixed point of $Y \mapsto \mathbb{1} \cup RY$

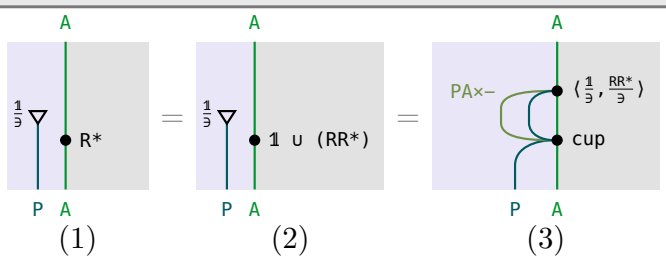
$$\mathbb{1} \sqsubseteq X;\ X^2 \sqsubseteq X;\ R \sqsubseteq X \Rightarrow R^* \sqsubseteq X \tag{6.7f}$$

a reflexive transitive $X$ containing $R$ is a prefixed point of $X \mapsto \mathbb{1} \cup RX$, so it contains $R^*$

| | | |
|---|---|---|
| 1 | $\mathbb{1} \cup (R\ X)$ | $\mathbb{1} \sqsubseteq X$ |
| 2 | $\sqsubseteq\ X \cup (R\ X)$ | $R \sqsubseteq X$ |
| 3 | $\sqsubseteq\ X \cup (X^2)$ | |
| 4 | $\sqsubseteq\ X$ | |

$$\frac{R^*}{\ni} = \langle \frac{\mathbb{1}}{\ni}, \frac{R\ R^*}{\ni} \rangle\ \mathrm{cup} \tag{6.7g}$$

the set of $R^*$-successors of $a$ is $a$ itself joined with the $R^*$-successors of its $R$-successors




| | | |
|---|---|---|
| 1 | $\frac{R^*}{\ni}$ | $\mathbb{1} \cup (R\ R^*) = R^*$ |
| 2 | $=\ \frac{\mathbb{1} \cup (R\ R^*)}{\ni}$ | $\frac{R \cup S}{\ni} = \langle \frac{R}{\ni}, \frac{S}{\ni} \rangle\ \mathrm{cup}$ |
| 3 | $=\ \langle \frac{\mathbb{1}}{\ni}, \frac{R\ R^*}{\ni} \rangle\ \mathrm{cup}$ | |

## §6.7.1. Computing closure

**R − ∅ = R** (6.7.1a)

taking nothing away leaves R

**R ∪ (S − R) = R ∪ S**

beside R, only the part of S outside R adds anything

**R − (S ∪ T) = R − S − T**

taking away S and then T takes away S∪T

**(R ∪ S) − T = (R − T) ∪ (S − T)**

taking T away from a union takes it away from each part

**monotonic(φ); monotonic(ψ)** (6.7.1b)

**⇒ (μX : φ(ψ(X))) = φ(μY : ψ(φ(Y)))**

the least fixed point of φ after ψ is φ applied to the least fixed point of ψ after φ

**S R* = (μX : S ∪ (X R))** (6.7.1c)

S followed by any number of R steps is the least X containing S and closed under a further R step

**R* S = (μX : S ∪ (R X))**

any number of R steps followed by S is the least X containing S and closed under an R step in front

**θ(∅,S) = S R*** (6.7.1d)

started with nothing found and S to explore, θ computes SR*

| | | |
|---|---|---|
| 1 | θ(∅,S) | θ(P,S) ≜ P ∪ (μX : S ∪ ((X R) − P)) |
| 2 | ≜ ∅ ∪ (μX : S ∪ ((X R) − ∅)) | (6.7.1a) R − ∅ = R |
| 3 | = ∅ ∪ (μX : S ∪ (X R)) | ∅ ∪ R = R |
| 4 | = (μX : S ∪ (X R)) | (6.7.1c) S R* = (μX : S ∪ (X R)) |
| 5 | = S R* | |

**θ(P,∅) = P** (6.7.1e)

with nothing left to explore, θ returns what it has found

| | | |
|---|---|---|
| 1 | θ(P,∅) | θ(P,Q) ≜ P ∪ (μX : Q ∪ ((X R) − P)) |
| 2 | ≜ P ∪ (μX : ∅ ∪ ((X R) − P)) | (μX : ∅ ∪ ((X R) − P)) = ∅ |
| 3 | = P ∪ ∅ | P ∪ ∅ = P |
| 4 | = P | |